COS120 - CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BÀI 2: TÌM KIẾM

MỤC TIÊU

- Hiểu và áp dụng được các phương pháp tìm kiếm.
- Cài đặt được các giải thuật tìm kiếm trên các dữ liệu thực tế.
- Lựa chọn được phương pháp tìm kiếm phù hợp cho từng bài toán cụ thể.

NỘI DUNG

- 2.1 - GIỚI THIỆU BÀI TOÁN TÌM KIẾM
- 2.2 - TÌM KIẾM TUYẾN TÍNH
- 2.3 - TÌM KIẾM NHỊ PHÂN

TÌM KIẾM

2.1 - GIỚI THIỆU BÀI TOÁN TÌM KIẾM

GIỚI THIỆU

- Tìm kiếm là công việc rất phổ biến và thường xuyên được thực hiện:
  - Tìm kiếm trong danh **bạ điện thoại**
  - Tìm kiếm sản phẩm theo ý muốn **trên trang web mua sắm**
  - Tìm kiếm một sinh viên theo **mã sinh viên trong cơ sở dữ liệu**
  - Tìm kiếm và sắp xếp các bài viết hoặc các tài khoản trên các mạng xã hội (Facebook, Instagram),…
- Áp dụng vào các lĩnh vực và tại các công ty lớn như:
  - Tìm kiếm trong máy học và trí tuệ nhân tạo: để tối ưu hóa các mô hình giúp hoạt động hiệu quả.
  - Netflix Recommendation System: để đề xuất các bộ phim cho người dùng.
  - Amazon Product Search: để hiển thị sản phẩm phù hợp nhất với từ khóa tìm kiếm.
  - Google Maps: giúp người dùng tìm kiếm địa điểm, tìm đường đi.

MÔ TẢ

- Tìm kiếm là quá trình xác định một đối tượng nào đó trong một tập các đối tượng.
- Nếu tìm thấy thì kết quả trả về là đối tượng tìm được hoặc **vị trí** của đối tượng trong tập đó.
- Việc tìm kiếm dựa theo các điều kiện hoặc **một trường** nào đó của đối tượng, trường này là khóa (key) của việc tìm kiếm.

CÁC VÍ DỤ:

- Tìm sinh viên có họ tên Nguyễn Văn Xuân trong danh sách sinh viên
- Tìm các điện thoại thỏa các điều kiện: phải là sản phẩm **iPhone**, có giá **dưới 20 triệu đồng**, bộ nhớ trong **dưới 128GB**.
- Cho mảng một chiều các số nguyên a có {2, 4, 8, 7, 8, 4, 9, 5}, với **x=8**. Hãy **tìm x** trong mảng a. Ta thấy x xuất hiện tại **vị trí 2 và 4** trong mảng a.

CÁC PHƯƠNG PHÁP TÌM KIẾM

- Có rất nhiều phương pháp tìm kiếm như: Tìm kiếm theo đám mây (Hashing), Tìm kiếm nhảy (Jump Search), Tìm kiếm tương tự (Similarity Search), Tìm kiếm trong cây (Tree-based Search), tìm kiếm đồng thời (Concurrent Search), Tìm kiếm vét cạn (Brute-force Search), tìm kiếm tuyến tính (Linear Search), tìm kiếm nhị phân (Binary Search),…

- Chúng ta chỉ tập trung vào hai phương pháp tìm kiếm cơ bản:

  - **TÌM KIẾM TUYẾN TÍNH (LINEAR SEARCH)**
  - **TÌM KIẾM NHỊ PHÂN (BINARY SEARCH)**

TÌM KIẾM

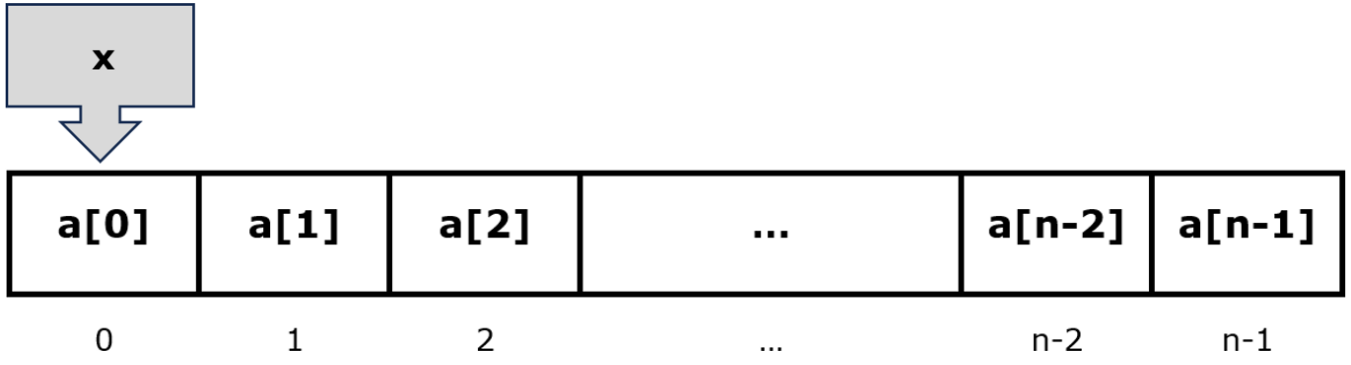
2.2 - TÌM KIẾM TUYẾN TÍNH

GIẢI THUẬT

Ý TƯỞNG

VÍ DỤ: Ta tiến hành tìm sự xuất hiện của **khóa x** trong mảng a.

- Thuật toán bắt đầu từ đầu mảng.
- So sánh khóa x lần lượt với từng phần tử của mảng a cho đến khi gặp được phần tử có giá trị trùng với khóa.
- Hoặc đã tìm hết mảng mà không thấy phần tử nào trùng với khóa.



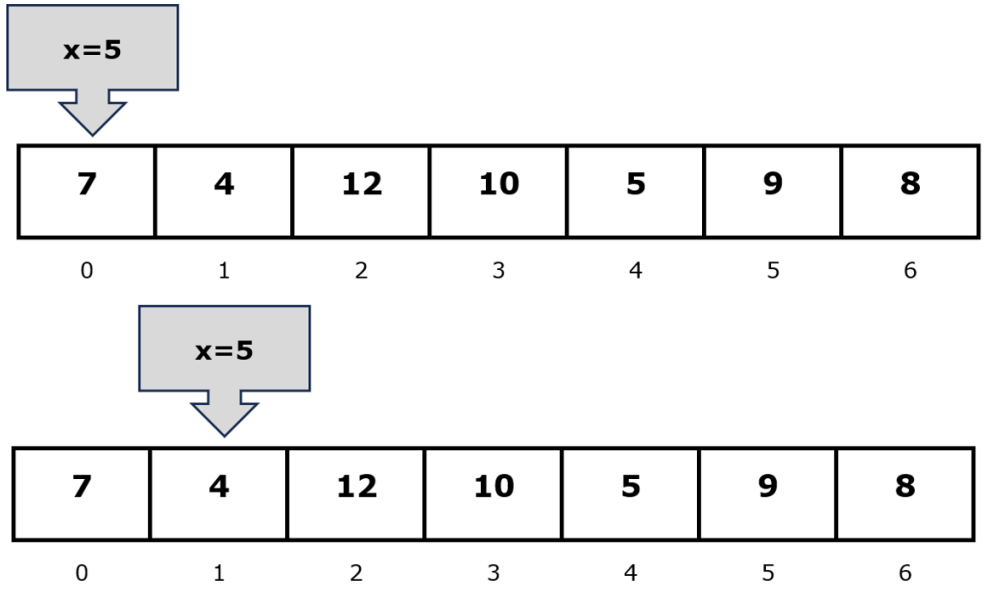
GIẢI THUẬT

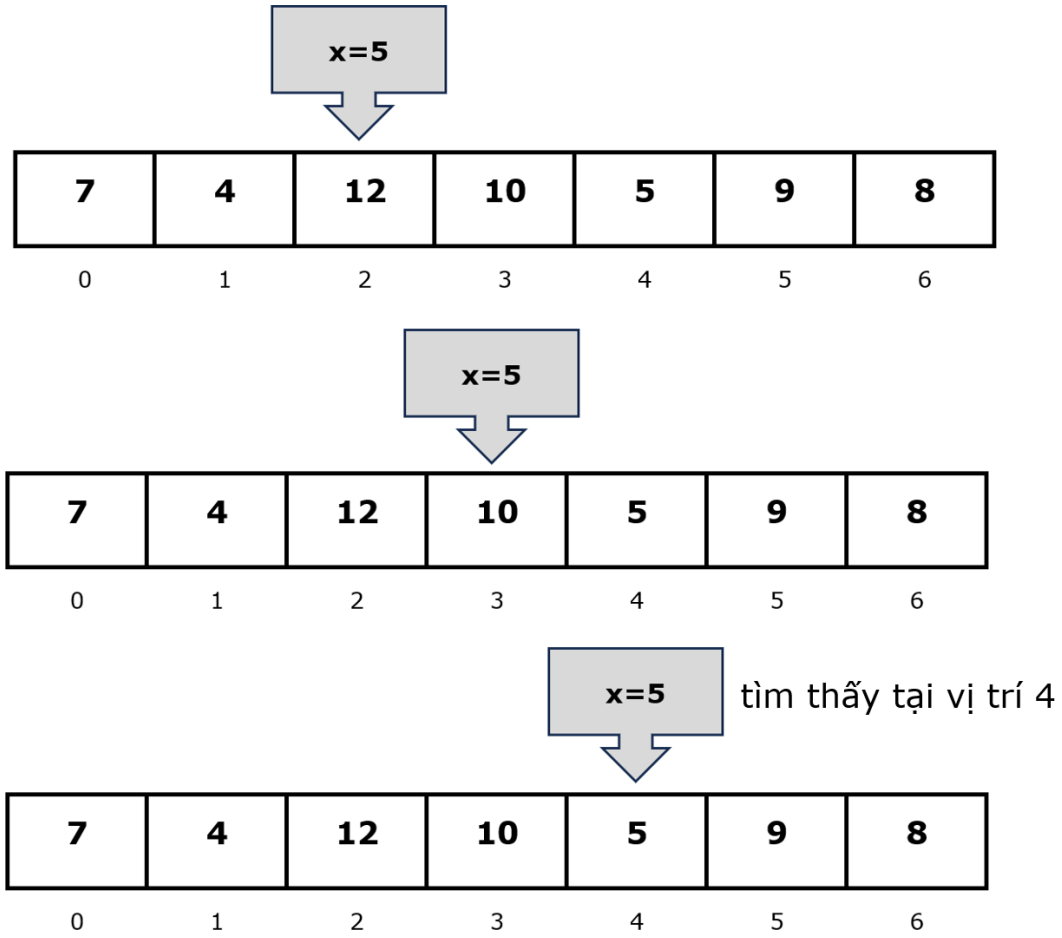
- Các bước tiến hành như sau:

```
Bước 1:
    ▪ i = 0; // bắt đầu từ phần tử đầu tiên của mảng
Bước 2: So sánh a[i] với x, có  2 khả năng :
    ▪ a[i] = x  : Tìm thấy thì dừng
    ▪ a[i] != x  :  Chuyển sang Bước 3.
Bước 3:
    ▪  i = i+1;       // xét phần tử kế tiếp trong mảng
    ▪  Nếu i>=n  : Hết mảng, không tìm thấy thì dừng
       Ngược lại: Lặp lại Bước 2.
```

VÍ DỤ MINH HỌA

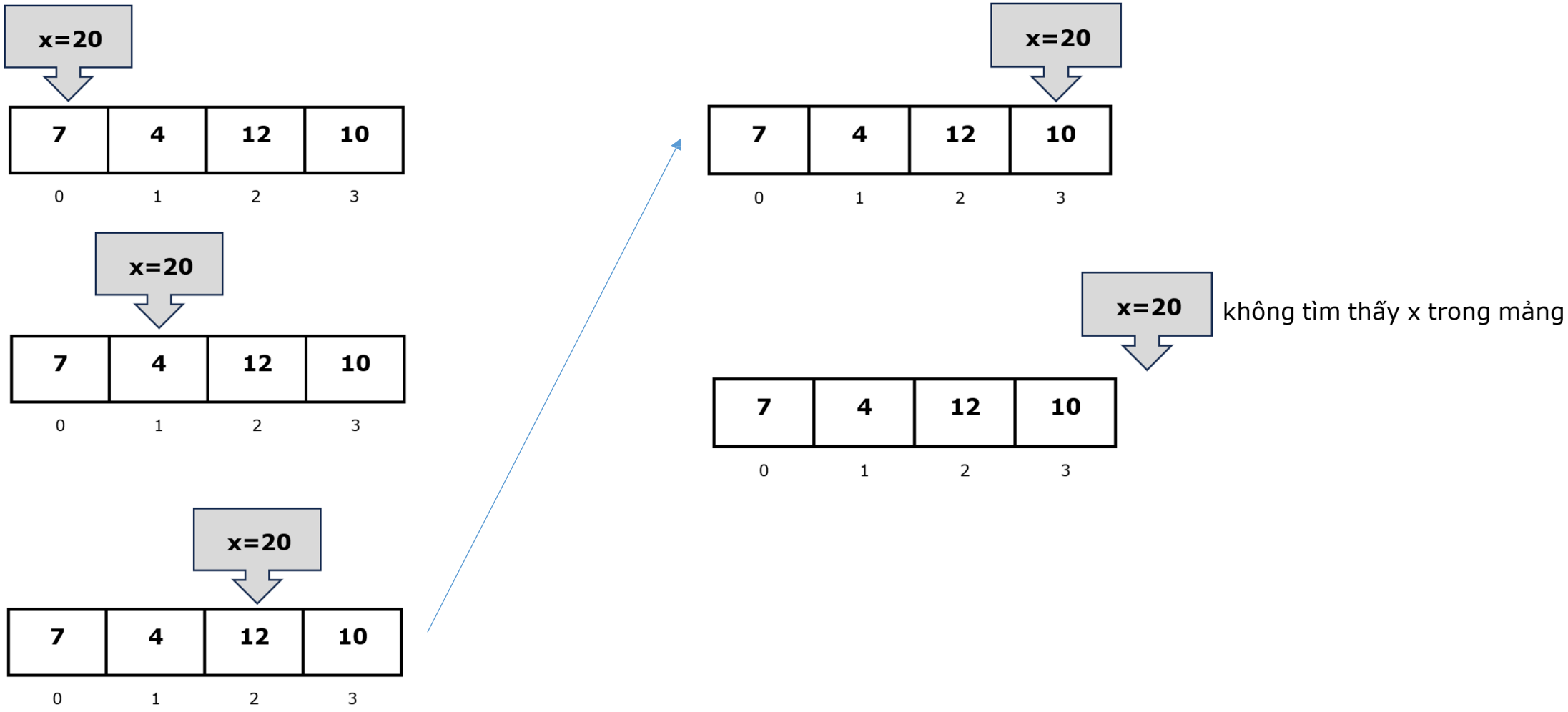
- Tìm x=5 trong mảng a có n=7 phần tử.





VÍ DỤ MINH HỌA

- Tìm x=20 trong mảng a có n=4 phần tử



CÀI ĐẶT

❑ CÀI ĐẶT BẰNG VÒNG LẶP FOR

```
int LinearSearch(int a[], int n, int x)
{
    int i;
    for(i=0; i<n ;i++)
        if(a[i]==x)
            return i;
    return -1;
}
```

❑ CÀI ĐẶT BẰNG VÒNG LẶP WHILE

```
int LinearSearch(int a[], int n, int x)
{
    int i=0;
    while ((i<n) && (a[i]!=x ))
        i++;
    if(i==n)
        return -1;// tìm hết mảng nhưng không có x
    else
        return i;// a[i] là phần tử có khoá x
}
```

CẢI TIẾN

NHẬN XÉT

Xét trường hợp cài đặt bằng vòng lặp **while** nói trên:

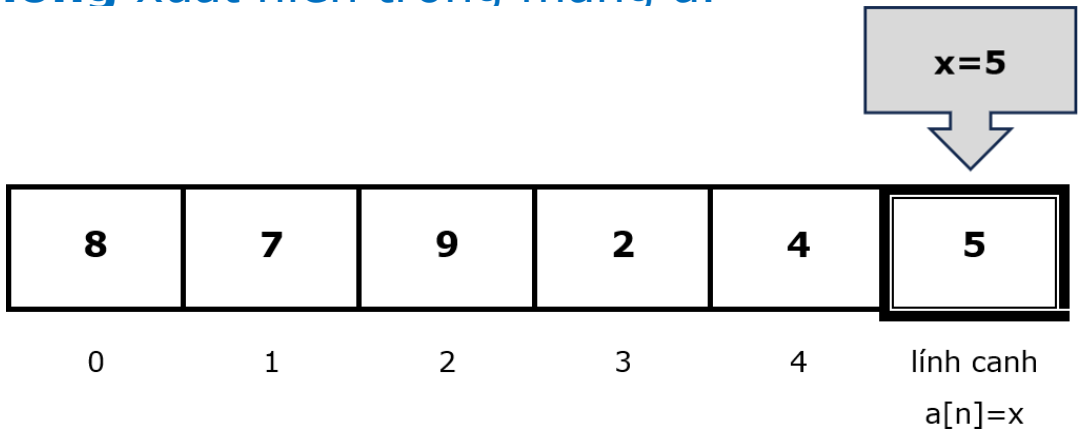
- Cứ mỗi lần lặp ta phải thực hiện kiểm tra hai điều kiện (i<n) và (a[i]!=x).
- Trong đó, kiểm tra điều kiện (a[i] !=x) là việc kiểm tra chính.

CẢI TIẾN

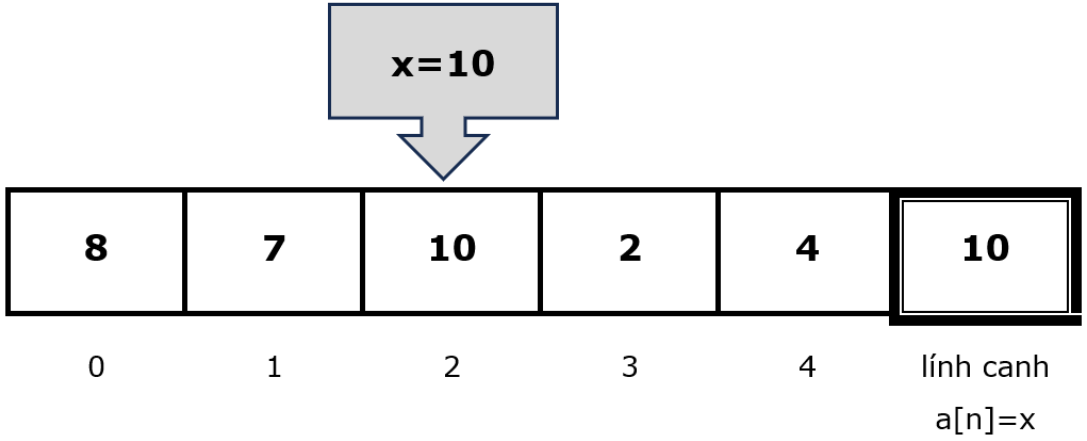
- Ta **đặt thêm một phần tử lính canh** có giá trị x vào cuối mảng.
- Như vậy bảo đảm luôn tìm thấy x trong mảng.
- Nếu **tìm thấy** x tại vị trí lính canh thì có nghĩa là x **không** xuất hiện trong mảng
- Nhờ lính canh, ta giảm bớt động tác kiểm tra điều kiện (i<n).

VÍ DỤ MINH HỌA

- Ví dụ: tìm x=5 trong mảng a có n=5 phần tử (có đặt lính canh). Tìm **thấy x** tại vị trí lính canh có nghĩa là x **không** xuất hiện trong mảng a.



- Ví dụ: tìm x=10 trong mảng a có n=5 phần tử (có đặt lính canh). Tìm thấy x **không** phải tại vị trí lính canh có nghĩa là x có xuất hiện trong mảng a



CÀI ĐẶT - CẢI TIẾN VỚI LÍNH CANH

```
int LinearSearch(int a[], int n, int x)
{
    int i=0;   // mảng gồm n phần tử từ a[0] đến a[n-1]
    a[n] = x; // thêm phần tử thứ n+1 làm lính canh
    while (a[i]!=x )
          i++;
    if (i==n) //nếu tìm thấy x tại vị trí lính canh
           return -1;      // nghĩa là không tìm thấy x
    else
          return i;  // tìm thấy x tại vị trí i
}
```

NHẬN XÉT

VỊ TRÍ XUẤT HIỆN CỦA x	SỐ LẦN SO SÁNH	ĐÁNH GIÁ
x ở tại vị trí đầu tiên của mảng	1	Đây là trường hợp tốt nhất
x xuất hiện ở cuối mảng hoặc không xuất hiện	n+1	Đây là trường hợp xấu nhất
x xuất hiện ở khoảng giữa của mảng	(n+1)/2	Đây là trường hợp trung bình

- Vậy giải thuật tìm kiếm tuyến tính có độ phức tạp tính toán là O(n).

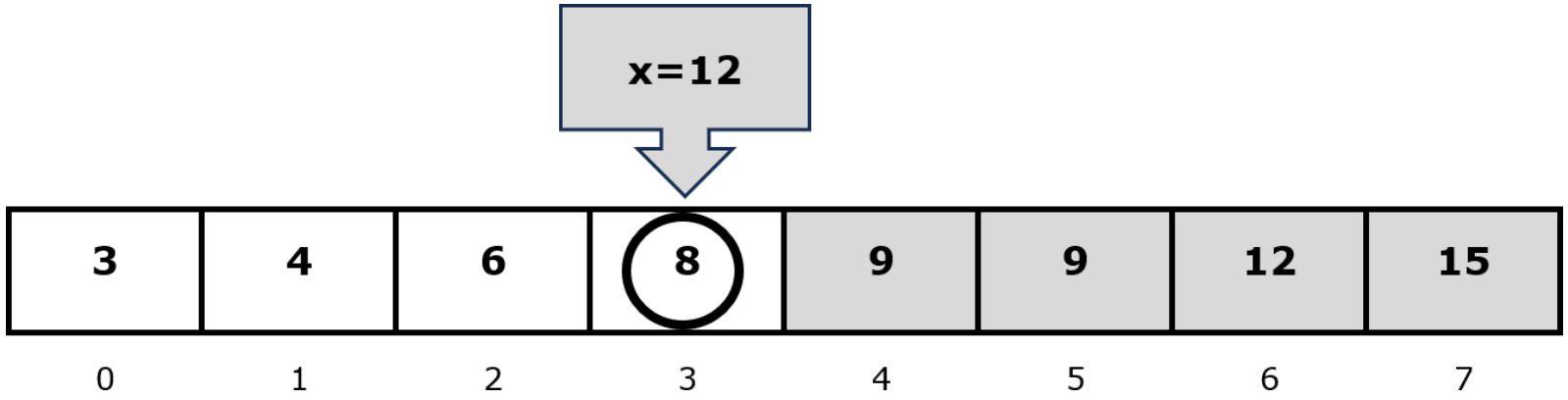
TÌM KIẾM

2.3 - TÌM KIẾM NHỊ PHÂN

GIẢI THUẬT

Ý TƯỞNG

- **Tìm kiếm phần tử x** trên danh sách **đã sắp xếp** (tăng hoặc giảm)
- Giải thuật tìm cách giới hạn phạm vi tìm kiếm sau mỗi lần so sánh x với một phần tử trong dãy.
- Ý tưởng của giải thuật là tại mỗi bước tiến hành **so sánh x** với phần tử nằm ở vị trí giữa của dãy tìm kiếm hiện hành
- Dựa vào kết quả so sánh này để quyết định **giới hạn dãy tìm kiếm ở bước kế tiếp** là nửa trên hay nửa dưới của dãy tìm kiếm hiện hành.



GIẢI THUẬT

❑ Các bước thực hiện

```
Bước 1: left = 0; right = n-1; //tìm kiếm trên tất cả các phần tử
Bước 2:
     mid = (left+right)/2;          // lấy mốc so sánh
     So sánh a[mid] với x, có  3 khả năng :
        ▪ a[mid] = x: Tìm thấy thì dừng
        ▪ a[mid]>x://tìm tiếp x trong dãy con a[left]..a[mid -1]
            right = mid - 1;
        ▪ a[mid]<x://tìm tiếp x trong dãy con a[mid +1]..a[right]
            left = mid + 1;
Bước 3:
     Nếu left <= right     //còn phần tử chưa xét, tìm tiếp.
            Lặp lại Bước 2.
     Ngược lại: Dừng //Đã xét hết tất cả các phần tử.
```

VÍ DỤ MINH HỌA

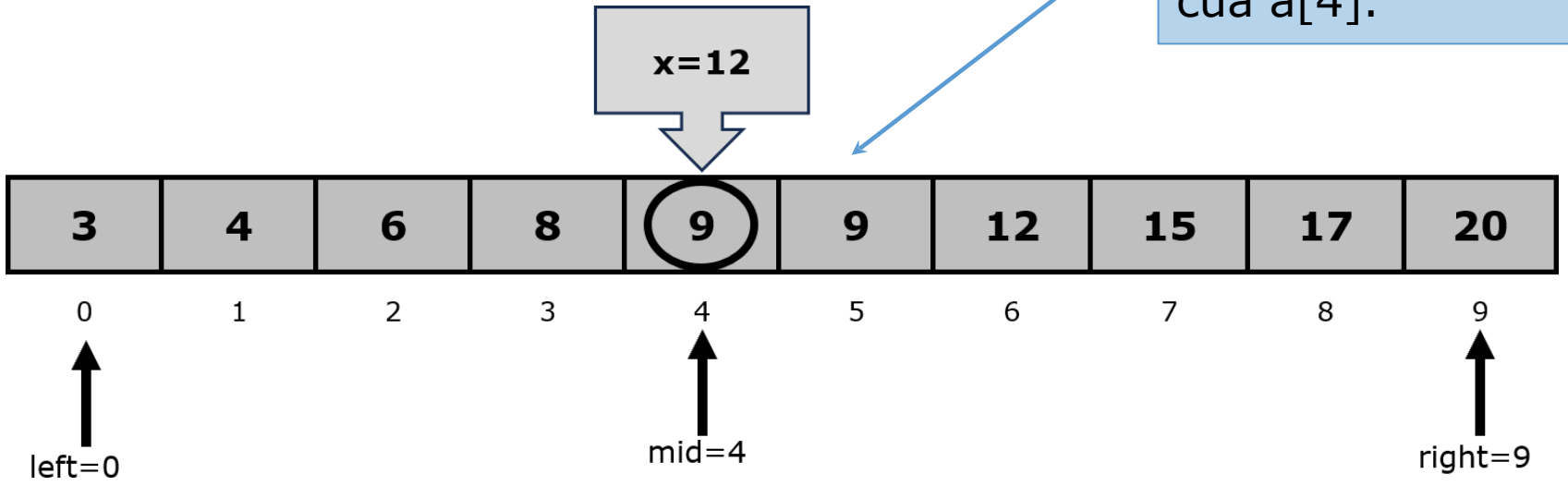
Cho mảng int đã sắp xếp tăng, có n=10 phần tử, tìm **x=12**

a = {3, 4, 6, 8, 9, 9, 12, 15, 17, 20}.

Khởi đầu: left = 0, right = n-1 = 9

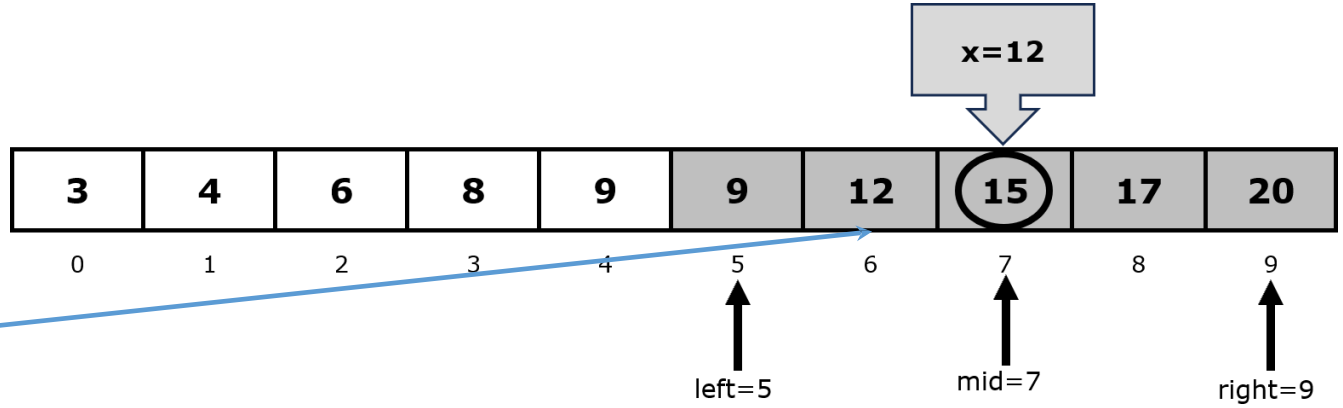
mid = (left+right)/2 = (0+9)/2 = 4

Do x>a[4] nên ở bước tiếp theo ta tìm x ở đoạn phía sau của a[4].

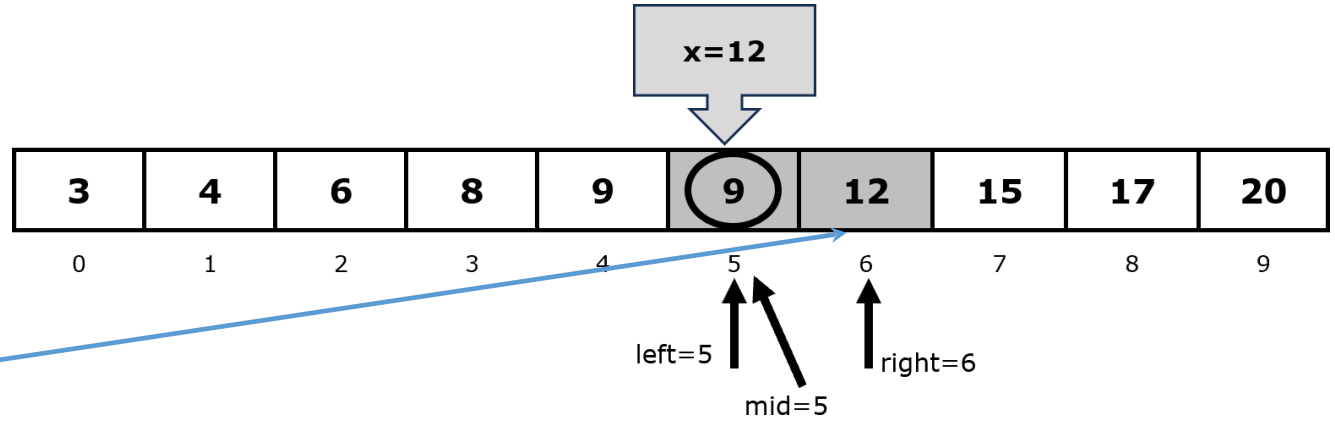


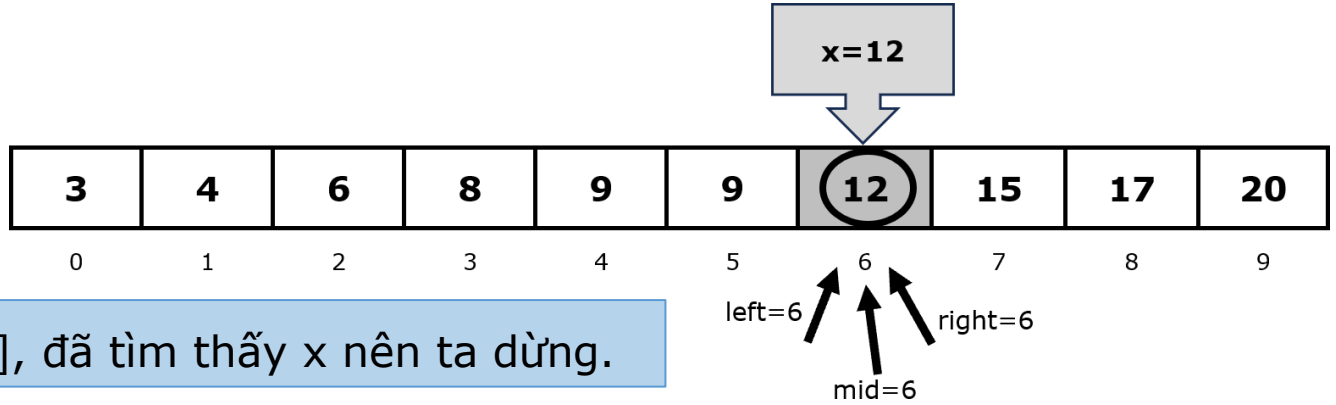
VÍ DỤ MINH HỌA

Do x<a[7] nên ở bước tiếp theo ta tìm x ở đoạn phía trước của a[7].



Do x>a[5] nên ở bước tiếp theo ta tìm x ở đoạn phía sau của a[5].





Do x=a[6], đã tìm thấy x nên ta dừng.

CÀI ĐẶT

```
int BinarySearch(int a[],int n, int x )
{
      int left = 0; right = n-1;
      int mid;
      do{
            mid = (left + right)/2;
            if (x == a[mid])
                        return mid;//Tìm thấy x tại mid
                  else if (x < a[mid])
                        right = mid -1;
                     else
                        left = mid +1;
      }while (left <= right);
      return -1; // tìm hết dãy mà không có x
}
```

NHẬN XÉT

VỊ TRÍ XUẤT HIỆN CỦA x	SỐ LẦN SO SÁNH	ĐÁNH GIÁ
x ở tại vị trí phần tử giữa mảng	1	Đây là trường hợp tốt nhất
x không xuất hiện trong mảng	$\log_2 n$	Đây là trường hợp xấu nhất
x xuất hiện ở một phía nào đó của mảng	$\log_2 n/2$	Đây là trường hợp trung bình

- Giải thuật **tìm kiếm nhị phân** tiết kiệm thời gian hơn rất nhiều so với giải thuật tìm kiếm tuyến tính. **Nó có độ phức tạp tính toán là $O(\log_2 n)$.**

BÀI TẬP

BÀI 1: Cho dãy số sau: 3 4 6 6 12 16 21 34 41 80. Hãy chạy bằng tay để cho biết vị trí tìm thấy và **số lần so sánh** để tìm được phần tử có giá trị x = 6 khi áp dụng giải thuật tìm kiếm tuyến tính và tìm kiếm nhị phân.

BÀI 2: Xây dựng giải thuật tìm kiếm phần tử có giá trị nhỏ nhất trong dãy số bằng cách dùng mã giả và lưu đồ.

BÀI 3: Viết chương trình tìm kiếm nhị phân cho trường hợp mảng đầu vào (kiểu int) đã sắp xếp giảm dần.

BÀI 4: Dựa vào code của hai giải thuật tìm kiếm tuyến tính và tìm kiếm nhị phân có trong tài liệu này. Hãy viết các **hàm đếm số phép so sánh** khi thực hiện tìm kiếm tuyến tính hoặc tìm kiếm nhị phân trên một dãy số đã sắp xếp tăng cho trước.

BÀI TẬP

BÀI 5: Viết chương trình thao tác trên mảng số nguyên int. Gồm có: hàm nhập mảng, hàm xuất mảng, hàm tìm kiếm tuyến tính, hàm tìm kiếm nhị phân, hàm kiểm tra mảng tăng dần hay chưa. Nếu mảng đã tăng thì gọi hàm tìm kiếm nhị phân để tìm giá trị x (x nhập từ bàn phím), ngược lại thì gọi hàm tìm kiếm tuyến tính.

BÀI 6: Cho cấu trúc Hàng hóa (mã hàng: char[9], tên hàng: char[41], giá: float). Viết chương trình thực hiện:

a) Nhập, xuất danh sách hàng hóa gồm n món hàng.

b) Tìm món hàng có mã là X bằng phương pháp tìm kiếm tuần tự (X nhập từ bàn phím)

c) Tìm món hàng có tên là Y bằng phương pháp tìm kiếm nhị phân. (Y nhập từ bàn phím).

d) Tìm món hàng có giá là V bằng phương pháp tìm kiếm nhị phân. (V nhập từ bàn phím).

Q&A!